

Tutorial Sheet – Technical Support System (V1–V3)

Programming Fundamentals 1

Topic: Library Classes, Random, ArrayList, HashMap and HashSet

Purpose: Consolidation of Collections and Multi-Class Program Design

Part A: Understanding the System Design

1. Classes and Responsibilities

The Technical Support System contains three main classes:

- InputReader
- Responder
- SupportSystem

Answer the following:

1. What is the responsibility of the InputReader class?
 2. What is the responsibility of the Responder class?
 3. What is the responsibility of the SupportSystem class?
 4. Why is it good design practice to separate these responsibilities into different classes?
-

2. Interface vs Implementation

Answer the following:

1. What is meant by the *interface* of a class?
 2. Give two examples of what is included in the interface.
 3. Give two examples of what is part of the implementation but not the interface.
-

Part B: Random Responses (V2)

3. Using Random

Consider the following line of code:

```
int index = randomGenerator.nextInt(responses.size());
```

Answer the following:

1. What does `responses.size()` return?
 2. What range of values could `index` contain?
 3. Why must we use `responses.size()` instead of a fixed number?
-

4. Working with ArrayList

The `Responder` class stores default responses in an `ArrayList<String>`.

1. Write code to declare and create an `ArrayList<String>` called `messages`.
 2. Add three different support messages to the list.
 3. Write a loop to print all messages in the list.
-

Part C: Context-Sensitive Responses (V3)

5. ArrayList vs HashMap

Answer the following:

1. Why is an ArrayList not suitable for storing key–value pairs?
 2. What type of data structure is used instead?
 3. Explain what a key–value pair means in the context of the support system.
-

6. Using HashMap

Write Java code to:

1. Declare and create a HashMap<String, String> called responseMap.
 2. Add the following key–value pair:
 - Key: "slow"
 - Value: "Try upgrading your hardware."
 3. Write a line of code to retrieve the response associated with the key "slow".
-

7. Understanding Map Behaviour

Answer the following:

1. What happens if you insert a second value using the same key in a HashMap?
 2. Can a HashMap contain duplicate keys?
 3. Is reverse lookup (finding a key from a value) easy with a HashMap? Explain briefly.
-

Part D: Tokenising Input and Using Sets

8. Splitting User Input

In Version 3, user input is split using:

```
String[] wordArray = inputLine.split(" ");
```

1. What does the `split(" ")` method do?
 2. What type of data structure is `wordArray`?
-

9. Using HashSet

The words from the input are stored in a `HashSet<String>`.

1. Why is a `HashSet` used instead of an `ArrayList`?
 2. What happens if the same word appears twice in the user input?
 3. Does a `HashSet` maintain insertion order?
-

10. Iterating Through a HashSet

The following code is used inside the `Responder` class:

```
Iterator<String> it = words.iterator();  
while(it.hasNext()) {  
    String word = it.next();  
    String response = responseMap.get(word);  
    if(response != null) {  
        return response;  
    }  
}
```

Answer the following:

1. What is the purpose of the iterator?
 2. What does `responseMap.get(word)` return if the word is not found?
 3. Why do we check `response != null`?
-

Part E: Understanding Program Flow

11. Main Loop Logic

In Version 3, the loop condition is:

```
while(!input.contains("bye"))
```

1. What does this condition check?
 2. Why is `contains("bye")` used instead of `startsWith("bye")`?
 3. What happens when the user types "bye"?
-

12. Extension Thinking Question

Explain, in plain English, how Version 3 of the Technical Support System works from start to finish.

Your answer should mention:

- Reading user input
 - Tokenizing the input
 - Looking up words in a `HashMap`
 - Returning a context-sensitive or default response
-